

PizzaWise.

Aggregator & Custom Pizza Ordering System

Build a full-stack web application that lets users construct a custom pizza, automatically locate nearby pizzerias, compare prices and delivery times, and place an order based on the best value. We care as much about **how** you build it as what you build.

1 • Core Requirements

01

Pizza Builder

Construct a custom pizza: select size, crust, sauce, and toppings through a clean, guided flow.

02

Smart Comparison Engine

Determine user location (auto or manual), fetch or simulate menus from nearby pizzerias, and rank the best value by price, distance, and ETA.

03

Data Persistence & Accounts

User authentication, profiles, favorite pizza configurations, and order history — all persisted.

04

UX / UI

A clean, intuitive, highly responsive modern interface. Polish matters.

2 • Technical & Cloud Architecture

Choose your own stack, defend your choices in the README.

CLOUD DEPLOYMENT

Deployed on Railway using Docker containers, with a decoupled, service-based setup.

Join our Railway workspace:

railway.com/invite/ACgOVgOb9Q3

PIZZERIA DATA API

Build against our hosted mock pizzeria API. Your API key arrives with the invitation email.

Docs: pizza-api-production-9995.up.railway.app/docs

DATABASE

A persistent SQL or NoSQL store with a deliberate schema and versioned migrations.

OBSERVABILITY

Structured logging, monitoring, and standard /health endpoints.

ARCHITECTURE

Decoupled, modular, and scalable under high concurrent load, clean code design principles throughout.

BONUS

CLI + Claude Skill: A small CLI that talks to your API (place an order, check status). And a Claude Skill that understands the CLI so Claude can order pizza through your system. Optional — skipping it costs nothing.

3 • Deliverables

1

GitHub Repository

With a detailed README.md covering architecture choices, setup instructions, and trade-offs.

2

Live Demo URL

Publicly accessible deployment we can click through.

3

System Architecture Diagram

One clear diagram of components, data flow, and infrastructure.

4

User Documentation

Clear, user-facing docs: what the app does and how to use it. Write for your users, not for engineers.